

Gerenciamento de Configuração de Software – AVALIAÇÃO N2

Disciplina: Gerência de Configuração de Software

Sistema: Gestão de Estoque e Caixa

Formato: Documentação técnica (PDF)

Aluno: Rayssa Alves e Wagner Gomes

Data: 18/11/2025

DOCUMENTO 1 — ITENS DE CONFIGURAÇÃO E ESTRUTURA DO PROJETO

1.1. Itens de Configuração (ICs)

Backend

Controllers

AuthController: Gerencia a autenticação. Fornece o endpoint `/api/auth/me` para retornar os dados do usuário atualmente logado.

MovimentacaoEstoqueController: Expõe os endpoints REST para registrar movimentações de estoque (entradas/ajustes) feitas por um Admin.

ProdutoController: Expõe os endpoints REST para o CRUD (Criar, Ler, Atualizar, Deletar) completo da entidade Produto.

UsuarioController: Expõe os endpoints REST para o CRUD (Criar, Ler, Atualizar, Deletar, Alterar Status) completo da entidade Usuário.

VendaController: Expõe os endpoints para registrar novas Vendas (função de Caixa), e consultar/filtrar o Relatório de Vendas (função de Relatório).

Services

MovimentacaoEstoqueService: Contém a regra de negócio para registrar uma movimentação, validando e atualizando o estoque do produto correspondente.

ProdutoService: Contém a regra de negócio para o CRUD de produtos, incluindo a validação de código único e o método centralizado `alterarEstoque`.

UsuarioService: Contém a regra de negócio para o CRUD de usuários (validação de e-mail único, etc.) e implementa a interface `UserDetailsService` do Spring Security para carregar o usuário durante a autenticação.

VendaService: Contém a regra de negócio principal do caixa. É responsável por validar o estoque dos itens, cálculos, orquestrar a baixa automática de estoque (chamando o `ProdutoService`) e implementar a lógica de filtragem complexa para os relatórios.

Entities

Usuario: Representa o usuário do sistema. Principais Atributos: `nomeCompleto`, `email` (único), `senha`, `perfil` (Enum ADMIN/OPERADOR), `ativo` (Boolean).

Nota: Implementa `UserDetails` para integração com Spring Security.

Produto: Representa um item disponível para venda no estoque. Principais Atributos: `codigo` (único), `nome`, `categoria`, `quantidadeEstoque`, `precoUnitario`.

Venda: Representa a transação de venda (o "cabeçalho" do recibo). Principais Atributos: `usuario` (quem vendeu), `valorTotal`, `valorRecebido`, `troco`, `dataHora`, `List<ItemVenda>`.

ItemVenda: Representa um item específico dentro de uma Venda (as "linhas" do recibo). Principais Atributos: `venda` (associação à Venda), `produto` (associação ao Produto), `quantidade`, `precoUnitario` (histórico do preço no momento da venda), `subtotal`.

MovimentacaoEstoque: Representa um registro no histórico de movimentações. Principais Atributos: `produto`, `usuario` (quem movimentou), `tipo` (Enum ENTRADA/AJUSTE), `quantidade`, `motivo`, `dataHora`.

Repositórios

UsuarioRepository: Interface Spring Data JPA para acessar os dados da entidade Usuario.

ProdutoRepository: Interface Spring Data JPA para acessar os dados da entidade Produto.

VendaRepository: Interface Spring Data JPA para acessar os dados da entidade Venda (incluindo a consulta customizada com JPQL para filtros de relatório).

ItemVendaRepository: Interface Spring Data JPA para acessar os dados da entidade ItemVenda.

MovimentacaoEstoqueRepository: Interface Spring Data JPA para acessar os dados da entidade MovimentacaoEstoque.

Arquivos de Configuração

SecurityConfig (no pacote config): Arquivo central do Spring Security. Responsável por configurar a autenticação (HTTP Basic), Definir o PasswordEncoder (não criptografa), definir as regras de autorização (permissões de rota) para ADMIN e OPERADOR em todos os endpoints da API

application.properties (ou .yml): Define a conexão com o banco de dados (H2), a porta do servidor (ex: 8080) e outras configurações do Spring Boot.

DTOs

ProdutoRequest / ProdutoResponse: Define os dados para criar/atualizar um produto (Request) e os dados retornados (Response).

UsuarioRequest / UsuarioResponse: Define os dados para criar/atualizar um usuário (Request), incluindo validações de senha, e os dados públicos do usuário retornados (Response), omitindo a senha.

VendaRequest / VendaResponse: Define os dados necessários para registrar uma venda (Request: lista de itens e valor recebido) e os dados completos da venda registrada (Response: total, troco, usuário, itens, etc.).

ItemVendaRequest / ItemVendaResponse: Define os dados de um item na VendaRequest (ID do produto, quantidade) e os dados de um item na VendaResponse (nome, preço, subtotal).

MovimentacaoEstoqueRequest / MovimentacaoEstoqueResponse: Define os dados para registrar uma movimentação (Request: produto, tipo, quantidade, motivo) e os dados do histórico de movimentação (Response).

Mappers

ProdutoMapper: Converte ProdutoRequest (DTO) em Produto (Entidade) e Produto (Entidade) em ProdutoResponse (DTO).

UsuarioMapper: Converte UsuarioRequest (DTO) em Usuario (Entidade) e Usuario (Entidade) em UsuarioResponse (DTO).

VendaMapper: Converte Venda (Entidade) em VendaResponse (DTO), utilizando o ItemVendaMapper para a lista de itens.

ItemVendaMapper: Converte ItemVenda (Entidade) em ItemVendaResponse (DTO), buscando dados do produto associado (como o nome).

MovimentacaoEstoqueMapper: Converte MovimentacaoEstoqueRequest (DTO) em MovimentacaoEstoque (Entidade) e MovimentacaoEstoque (Entidade) em MovimentacaoEstoqueResponse (DTO).

Enums

PerfilUsuario: Define os tipos de papéis de usuário (ADMIN, OPERADOR) para controle de acesso.

TipoMovimentacao: Define os tipos de movimentação de estoque (ENTRADA, AJUSTE_POSITIVO, AJUSTE_NEGATIVO).

Exceptions

BusinessException: Exceção customizada para erros de regra de negócio (ex: "Estoque insuficiente").

ResourceNotFoundException: Exceção customizada para quando um recurso (ex: Produto com ID 5) não é encontrado no banco.

EmailConflictException: Exceção customizada para a tentativa de cadastro de um e-mail que já existe.

ApiError: Classe record que define a estrutura JSON padronizada para as respostas de erro da API.

ApiExceptionHandler: Classe global (@RestControllerAdvice) que intercepta exceções (como BusinessException, ValidationException, etc.) e as converte em uma resposta ApiError padronizada com o status HTTP correto.

Frontend

Componentes

LoginComponent: Tela de login que chama o AuthService para autenticar.

NavbarComponent: Menu de navegação superior, exibe links de acordo com o perfil do usuário (ADMIN/OPERADOR).

ProdutoListComponent / ProdutoFormComponent: Telas de listagem e formulário para o CRUD de Produtos.

UsuarioListComponent / UsuarioFormComponent: Telas de listagem e formulário para o CRUD de Usuários.

MovimentacaoListComponent / MovimentacaoFormComponent: Telas para registrar e listar o histórico de movimentações de estoque.

VendaListComponent / VendaFormComponent: Telas de Relatório de Vendas (listagem) e a tela de PDV (formulário de registro de venda).

Services

AuthService: Gerencia o login (Basic Auth), logout e armazena o token/usuário na sessionStorage. Consome o endpoint /api/auth/me.

ProdutoService: Encapsula as chamadas HTTP para o CRUD de produtos. Consome /api/produtos/**.

UsuarioService: Encapsula as chamadas HTTP para o CRUD de usuários. Consome /api/usuarios/**.

VendaService: Encapsula as chamadas HTTP para listar e registrar vendas. Consome /api/vendas/**.

MovimentacaoEstoqueService: Encapsula as chamadas HTTP para listar e registrar movimentações. Consome /api/movimentacoes-estoque/**.

Rotas

app.routes.ts: Arquivo central que mapeia os caminhos da URL (ex: produtos, produtos/:id, usuarios/novo, vendas, login, etc.) para os seus respectivos componentes. A rota raiz (/) redireciona para /login.

Formulários / Guards / Interceptors

authInterceptor: (Interceptor) Intercepta todas as requisições HTTP e anexa automaticamente o token de autenticação (Basic Auth) no header Authorization.

Validação (Client-side): Implementada nos métodos validarCampos() dentro dos componentes de formulário (ex: usuario-form.component.ts, produto-form.component.ts), garantindo que os dados sigam as regras de negócio antes de serem enviados à API.

Formularios (Template-driven): O projeto utiliza FormsModule e ngModel para fazer a ligação (binding) dos dados entre os formulários HTML e as variáveis dos componentes.

Banco de Dados

Scripts SQL / Migrations

Não há arquivos de migration SQL manuais. A estrutura do banco de dados (schema) é gerenciada automaticamente pelo Spring Data JPA / Hibernate, com base na configuração spring.jpa.hibernate.ddl-auto=update no application.properties.

O schema é gerado a partir das classes de Entidade Java

Tabelas

usuarios: Armazena os dados dos usuários do sistema. Campos principais: id, nome_completo, email (unique), senha, perfil, ativo.

produtos: Armazena o catálogo de produtos e seu estoque. Campos principais: id, codigo (unique), nome, categoria, quantidade_estoque, preco_unitario.

vendas: Armazena o "cabecalho" de cada transação de venda. Campos principais: id, usuario_id (FK), valor_total, valor_recebido, troco, data_hora.

itens_venda: Armazena os itens individuais de cada venda. Campos principais: id, venda_id (FK), produto_id (FK), quantidade, preco_unitario, subtotal.

movimentacoes_estoque: Armazena o histórico de entradas e ajustes de estoque. Campos principais: id, produto_id (FK), usuario_id (FK), tipo, quantidade, motivo, data_hora.

Documentação

Documento de Requisitos e Arquitetura: O arquivo Sistema_de_Gesto_de_Estoque_e_Caixa, que define o escopo do projeto, regras de permissão, módulos, diagramas conceituais de arquitetura e o fluxo de login.

Documento de Avaliação (GCS): O arquivo AVALIAO_N2_GERNCIA_DE_CONFIGURAO_DE_SOFTWARE, que define os requisitos para esta documentação de Gerência de Configuração.

Template de Documentação (GCS): O arquivo TEMPLATE_COMPLETO_AVALIAO_N2, usado como base para a entrega.

Documentação de Build (Spring): HELP.md (arquivo padrão do Spring Boot com links para documentação).

Documentação de Build (Angular): README.md (arquivo padrão do Angular CLI com instruções de build e desenvolvimento).

Configurações

Backend: application.properties (Arquivo principal de configuração do Spring Boot), e SecurityConfig.java (Item de configuração que define todas as regras de segurança, permissões de rota e o provedor de autenticação)

Frontend: [environment.ts](#) (Define as variáveis de ambiente), angular.json (Arquivo de configuração do Angular CLI que define a estrutura do projeto), e [app.config.ts](#) (Arquivo de configuração da aplicação Angular)

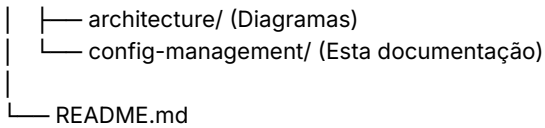
Build: pom.xml (Backend) (Define as dependências do projeto Maven, a versão do Java, e como o projeto deve ser empacotado), package.json (Frontend) (Define as dependências do projeto NPM e os scripts de build)

Descrição da Organização:

A estrutura proposta organiza o sistema em quatro diretórios principais na raiz: backend, frontend, database e docs. Essa abordagem de "monorepo" facilita a Gerência de Configuração, mantendo uma fonte única de verdade para todo o projeto. Ela isola claramente as responsabilidades: backend/ contém toda a lógica de negócio e API (Spring Boot), frontend/ contém toda a aplicação de interface do usuário (Angular), database/ centraliza a evolução do schema (migrações) e docs/ armazena a documentação (como os PDFs de requisitos). Essa separação permite que as equipes trabalhem de forma independente, facilita a manutenção e torna o versionamento de cada componente (API, UI, DB) mais claro e controlado.

1.2. Estrutura do Projeto (Conceitual)

```
sistema-estoque-caixa/
├── backend/
│   ├── src/main/java/com/example/SistemaRayssaWagner/SistemaGestaoEstoqueCaixa/
│   │   ├── config/
│   │   ├── controller/
│   │   ├── dto/
│   │   ├── entity/
│   │   ├── enums/
│   │   ├── exceptions/
│   │   ├── mapper/
│   │   ├── repository/
│   │   └── service/
│   ├── src/main/resources/
│   │   └── application.properties
│   ├── src/test/
│   └── pom.xml
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/
│   │   │   ├── core/ (Guards, Interceptors)
│   │   │   ├── models/
│   │   │   └── services/
│   │   ├── environments/
│   │   └── ...
│   └── package.json
├── database/
│   ├── migrations/ (Para DDL versionado)
│   └── seed/ (Para dados iniciais)
└── docs/
    └── requirements/ (PDFs de requisitos)
```



DOCUMENTO 2 — BASELINES E VERSIONAMENTO

2.1. Baseline BL0 – Inicial

Data: 03/11/2025

Descrição:

Marco inicial do projeto, onde a estrutura de pastas do monorepo foi definida. Inclui os projetos "esqueleto" do backend (Spring Boot) e do frontend (Angular), com suas dependências principais configuradas, mas sem nenhuma funcionalidade de negócio implementada. O documento de requisitos também é incluído como o artefato que guiará o desenvolvimento.

Conteúdo da BL0:

backend/ (Projeto Spring Boot)

frontend/ (Projeto Angular)

docs/ (Pasta para documentação)

README.md (Documento raiz do projeto)

Itens de configuração presentes:

Estrutura de diretórios do "monorepo".

Arquivos de build (pom.xml, package.json).

Configurações de ambiente (application.properties, environment.ts).

Documentação de Requisitos.

Critério de Estabilidade:

1. O projeto backend (Spring) inicia (spring-boot:run) sem erros e conecta-se com sucesso ao banco H2.
2. O projeto frontend (Angular) inicia (ng serve) sem erros e exibe a página principal.
3. As dependências de build (pom.xml, package.json) estão instalando corretamente (Maven e NPM).

2.2. Baseline BL1 – Funcional

Data: 18/11/2025

Descrição:

Marco principal que representa a versão funcional e completa do sistema. Todos os módulos principais (Login, Usuários, Produtos, Movimentações, Caixa e Relatórios) estão implementados e integrados. O controle de acesso por perfil (ADMIN/OPERADOR) está funcional, com o backend protegendo os endpoints e o frontend reagindo a eles.

Requisitos Entregues:

1. RF-Login: Autenticação de usuários (ADMIN e OPERADOR) via e-mail e senha.
2. RF-Usuarios: CRUD completo de Usuários, restrito ao perfil ADMIN.
3. RF-Produtos: CRUD completo de Produtos, restrito ao perfil ADMIN.
4. RF-Movimentacao: Registro de entradas e ajustes de estoque, restrito ao perfil ADMIN.
5. RF-Caixa: Registro de vendas (PDV) para OPERADOR, com cálculo de troco e baixa automática de estoque.
6. RF-Relatorios: Listagem de vendas realizadas, acessível por ambos os perfis.

Funcionalidades Implementadas:

1. Backend: API REST completa com todas as regras de negócio implementadas nos Services (ex: VendaService, ProdutoService).
2. Backend: Endpoint de consulta de vendas (/api/vendas) com lógica de filtragem por usuário, data e valor.
3. Backend: Segurança de API implementada com Spring Security, definindo permissões de rota por perfil (ADMIN/OPERADOR).
4. Frontend: Interface completa no Angular com PrimeNG, permitindo o gerenciamento (CRUD) de Produtos e Usuários.
5. Frontend: Tela de PDV (Caixa) funcional, com adição de itens, cálculo de total/troco e validação de estoque em tempo real.
6. Frontend: Gerenciamento de sessão (Login/Logout) e interceptor HTTP (authInterceptor) para envio do token de autenticação.

Critério de Estabilidade:

- 1. Todos os fluxos principais (Login, CRUD Usuários, CRUD Produtos, Registrar Venda, Registrar Movimentação) estão funcionando sem erros críticos.
- 2. A baixa de estoque após uma venda (RF-Caixa) é refletida corretamente na listagem de produtos (RF-Produtos).
- 3. As regras de permissão estão sendo aplicadas (Ex: Operador não consegue acessar a tela de "Usuários").
- 4. As validações de negócio do backend (ex: "Estoque insuficiente", "E-mail já cadastrado") estão sendo corretamente tratadas e exibidas no frontend.

2.3. Política de Versionamento

Estratégia Adotada:

Versionamento Semântico (Semantic Versioning)

Regras de Versão:

MAJOR (X.0.0): Será incrementado para mudanças que quebram a compatibilidade da API (breaking changes) ou quando o projeto atingir um marco funcional principal.

MINOR (0.X.0): Será incrementado quando novas funcionalidades forem adicionadas ao sistema (ex: um novo "Relatório de Estoque") mantendo a compatibilidade com as versões anteriores da API.

PATCH (0.0.X): Será incrementado para correções de bugs, ajustes de performance ou pequenas melhorias que não adicionam novas funcionalidades e mantêm a compatibilidade.

Tabela de Exemplos:

Versão	Descrição	Tipo
[v0.1.0]	BL0	Inicial
[v1.0.0]	BL1	MAJOR
[v1.1.0]	Adição da funcionalidade de filtros no Relatório de Vendas	MINOR
[v1.1.1]	Correção de um bug no cálculo de estoque no VendaService	PATCH

Relação Baselines → Versionamento:

BL0 → v0.1.0

BL1 → v1.0.0

DOCUMENTO 3 — RASTREABILIDADE + RFC

3.1. Matriz de Rastreabilidade

ID	Requisito	Arquivo(s) Relacionado(s)	Observação
[RF-01]	Autenticação de Usuário (Login)	Backend: SecurityConfig.java, UsuarioService.java, AuthController.java Frontend: login.component.ts, auth.service.ts, auth.interceptor.ts	O UsuarioService implementa UserDetailsService para o Spring Security. O AuthService (front) usa Basic Auth para se autenticar no endpoint /me.
[RF-02]	Manter Usuários (CRUD Admin)	Backend: UsuarioController.java, UsuarioService.java, Usuario.java, UsuarioRepository.java Frontend: usuario-list.component.ts, usuario-form.component.ts, usuario.service.ts	As rotas /api/usuarios/** são protegidas para ADMIN no SecurityConfig.java.
[RF-03]	Registrar Venda (Caixa)	Backend: VendaController.java, VendaService.java, VendaRequest.java, ProdutoService.java Frontend: venda-form.component.ts, venda.service.ts	A baixa de estoque é orquestrada no VendaService, que chama produtoService.alterarEstoque. Acesso restrito ao OPERADOR.
[RF-04]	Movimentação	Backend: MovimentacaoEstoqueController.java,	Acesso restrito ao ADMIN. O

	de Estoque (Admin)	MovimentacaoEstoqueService.java, ProdutoService.java Frontend: movimentacao-form.component.ts, movimentacao-list.component.ts	MovimentacaoEstoqueService usa o ProdutoService para alterar o estoque.
[RF-05]	Relatórios (Consulta de Vendas)	Backend: VendaController.java, VendaService.java, VendaRepository.java (método findByFilters) Frontend: venda-list.component.ts	Acesso permitido a ADMIN e OPERADOR. O backend suporta filtros, que é um requisito do PDF.

3.2. RFC – Solicitação de Mudança

Código da RFC: [RFC-001]

1. Descrição da Mudança:

A mudança consiste em alterar o serviço movimentacao-estoque.service.ts para que ele pare de enviar um valor fixo (hardcoded) "1" como usuarioid ao registrar uma movimentação. O serviço deverá injetar o AuthService para obter o ID do usuário autenticado na sessão e enviá-lo dinamicamente, da mesma forma que o venda.service.ts já faz.

2. Justificativa:

Atualmente, todas as movimentações de estoque (entradas, ajustes) estão sendo registradas incorretamente para o Usuário de ID 1, independentemente de qual administrador esteja logado. Isso corrompe a rastreabilidade do histórico de movimentações, que é um requisito funcional.

3. Impacto nos Itens de Configuração:

Item	Tipo de Alteração
frontend/src/app/services/movimentacao-estoque.service.ts	Modificação: Injetar AuthService no construtor. Alterar o método registrar() para usar o ID do authService.getUsuarioLogado() ao construir os HttpParams.

4. Riscos:

Risco: Risco baixo.

Mitigação: A lógica de injeção do AuthService e obtenção do ID do usuário logado já foi implementada e validada com sucesso em outro serviço (venda.service.ts). A mudança consiste apenas em replicar este padrão testado.

5. Status:

Aprovada

6. Versão de Implementação: [v1.0.1]

7. Data: 13/11/2025